第 11 章推導解答

DS11-E01 • B-tree 插入與刪除

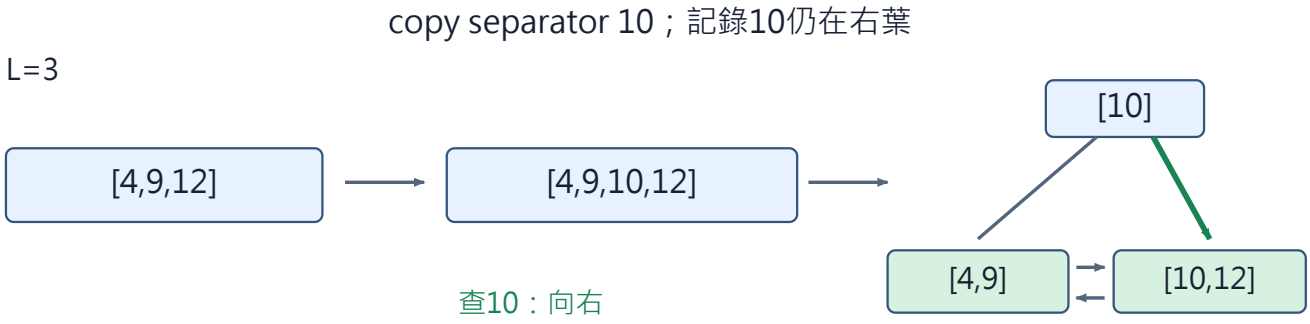
10、20、5 後根為 [5,10,20]。插 6 前分裂，提升 10，兩葉 [5]、[20]，6 放左葉成 [5,6]。12、30 依序放右葉成 [12,20,30]。插 25 前分裂該葉，提升 20，根 [10,20]，葉 [5,6]、[12]、[25,30]。刪 12 前，中葉只有 t−1=1 鍵，左兄弟有 2 鍵可借。6 上升替代父的 10，10 下移中葉，然後刪 12。結果根 [6,20]，葉 [5]、[10]、[25,30]。各非根葉有 1 至 3 鍵，根有兩鍵三子，所有葉深度 1。

DS11-E02 • 高度與 I/O

深度 d≥1 至少 2t^(d−1) 個節點，各至少 t−1 鍵，故總鍵至少 1+2(t−1)(1+t+…+t^{h−1})=2t^h−1。t=64 時，h=3 的下限為 524,287，h=4 的下限為 33,554,431，因此最大允許整數 h=3。單用最小容量下界沒有排除 h=2；但高度 2 的最大鍵數為 (2t)^{3}−1=128³−1=2,097,151，可容納 1,000,000，所以 h=2 也可能。高度是邊數，根葉路徑頁數為 h+1，另視根是否快取扣除讀取。

DS11-E03 • B+ 的 separator 與刪除

插入後暫時 [4,9,10,12]，分裂為 [4,9] 與 [10,12]，父複製 separator 10，10 仍在右葉。左葉 next 指向右葉，若雙向鏈則右葉 prev 指回左葉。查 10 依等於往右原則讀到記錄。另一個 L=2 狀態刪 10 後，右葉成 [12]，仍達最低 ceil(2/2)=1，不需借位合併，葉鏈不改。因本教材 separator 精確等於右子樹最小鍵，根鍵改成 12。若不更新就不符合定義，即使某些查詢仍可能恰好成功。



DS11-E04 • 範圍查詢成本的前提

題設M≥3保證a=ceil(M/2)≥2，對數底數合法。h≥1時根至少2子，以下每個內頁至少a子，葉數至少2a^(h−1)，故n≥2ell a^(h−1)、h≤1+log_a(n/(2ell))；h=0另付一頁。根葉定位O(h+1)，除首尾部分頁外，每個掃過葉至少貢獻ell筆，故索引I/O為O(h+1+k/ell)。若M、L皆Θ(B)、固定大小entry／record且一節點一頁，便可化成O(1+log_B n+k/B)。分支條件控制定位，葉最低容量條件控制掃描，兩者不能混為一個假設。

反例任取q≥3，M=3、L=B=2^q，每個內頁2子，2^q葉各滿B筆。$n=B^2$，h=q，查最右鍵需q+1頁，但log_B n=2；僅有L=Θ(B)不保證內頁分支隨B成長。

若完整記錄散在D個資料頁，再加D，最壞D=k；依鍵序聚集且每頁Θ(B)筆時，D可界為O(1+k/B)。不論I/O，產生k筆輸出需Ω(k)基本工作。

本章頁模型把(child_page_id,cached_min)存同一父頁，只有改動子頁將新min沿(page_id,index)祖先stack回報；不重讀全部子頁。內頁分裂搬完整entry，不保存／改寫子頁的磁碟parent。於是每層常數頁I/O，更新O(h+1)；若把min或parent放在各個子頁，單次refresh／分裂可多Θ(M)頁存取。頁內搬移仍可O(M)，不能把它當O(1)CPU工作。

DS11-E05 • order 定義不可混用

minimum degree t 必須是整數且最多子數為 2t，所以無法令 t=2.5。若甲另要求最少 ceil(m/2)=3 子，最大 5 子，非根鍵為 2 至 4；乙 t=2 為 1 至 3 鍵，t=3 為 2 至 5 鍵，都不完全相同。甲若以溢位後 5 鍵分裂，可提升中位，左右各 2 鍵；乙 top-down 分裂的是 2t−1 鍵滿節點，左右各 t−1。插入時機、滿容量與分裂分配必須成套指定，不能只替換一個符號。

DS11-E06 • B+ 缺額、精確separator與範圍

非根葉至少ceil(3/2)=2筆。刪10後中葉只剩[12]，兩兄弟各2筆不能借；與左[4,9]合成[4,9,12]，重接葉鏈為[4,9,12]↔[14,16]。父從三子減成兩子，separator只剩右最小14，根[14]，佔用合法。

刪14後右葉[16]缺額；左葉有3筆，借其最大12到右葉最前，得[4,9]↔[12,16]。兩葉各2筆，根separator由14更新12。查[9,14]先由根12向左，定位9，輸出9；沿next進右葉輸出12，讀16發現超過14而停止，答案[9,12]。查詢讀到終止用的16不等於把16輸出。

再刪4，左葉[9]缺額，右葉2筆不能借，合併為[9,12,16]。父只剩一子且無separator，縮根成該葉；根葉3筆合法，所有記錄都在同一頁。原父separator12可移除，因它只是葉中記錄12的路由副本；不把父separator再加進葉，否則會重複12。B-tree的父鍵則本身是資料，合併時必移入子節點；直接丟棄會遺失記錄。兩套規則不可混用。

DS11-E07 • 內頁修復完整追蹤

> 非根葉至少 2 筆，非根內頁至少 2 個 child。刪 1 後 A=[3]，B=[5,7] 沒有餘額。合併得 AB=[3,5,7]，保留 A 的頁 ID、釋放 B；令 AB.next=C、C.prev=AB。P.entries 變成 [(AB,3)]，只有 1 個 child，因此仍須向上修復。
>
> 第一種初態的 Q 有 3 個 child，可將 (C,9) 移到 P 尾端。P.entries=[(AB,3),(C,9)]，separator=[9]；Q.entries=[(D,13),(E,17)]，separator=[17]。根 entries=[(P,3),(Q,13)]，separator=[13]。各非根內頁恰 2 個 child；葉 AB 有 3 筆，其餘葉各 2 筆。葉鏈為 AB↔C↔D↔E，內頁借位不改此鏈。查 9 在 R 因 9<13 走 P，再因 9 等於 P 的 separator 而走 C，讀到記錄 9。

階段	P.entries	Q.entries	R.entries
初始	(A,1),(B,5)	(C,9),(D,13),(E,17)	(P,1),(Q,9)
葉合併後	(AB,3)	不變	(P,3),(Q,9)
借 child 後	(AB,3),(C,9)	(D,13),(E,17)	(P,3),(Q,13)

第二種初態沒有 E，Q 只有 (C,9),(D,13)，不能出借。合併內頁得 P.entries=[(AB,3),(C,9),(D,13)]，3 個 child 未超過 M=4；P 的 separator=[9,13]。R 只剩 (P,3)，所以 root_id=P，釋放 Q、R，高度降為 1。葉鏈 AB↔C↔D，記錄為 [3,5,7,9,11,13,15]；原有 8 筆恰刪 1 筆。

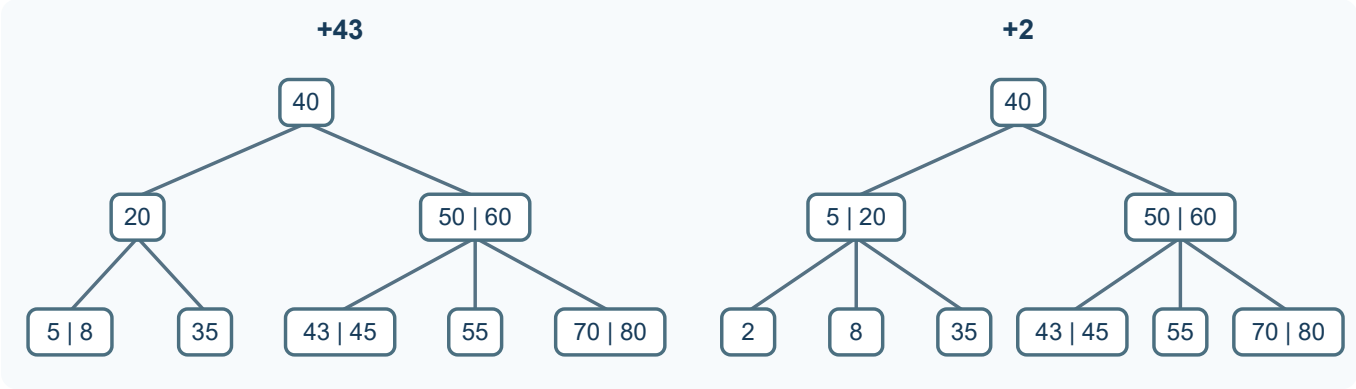
查 [7,14] 在根 [9,13] 走 AB，先輸出 7；再讀 C 的 9、11，D 的 13，檢查 15>14 後停止。輸出 [7,9,11,13]。新根有 3 個 child、2 個 separator；9 是路由副本，真正記錄 9 已在 C。若再把舊根 separator 9 當資料插入葉，就會重複記錄 9；正確合併不增加任何葉記錄。B-tree 的父鍵本身承載記錄，須移入合併子頁，不能與此規則混用。

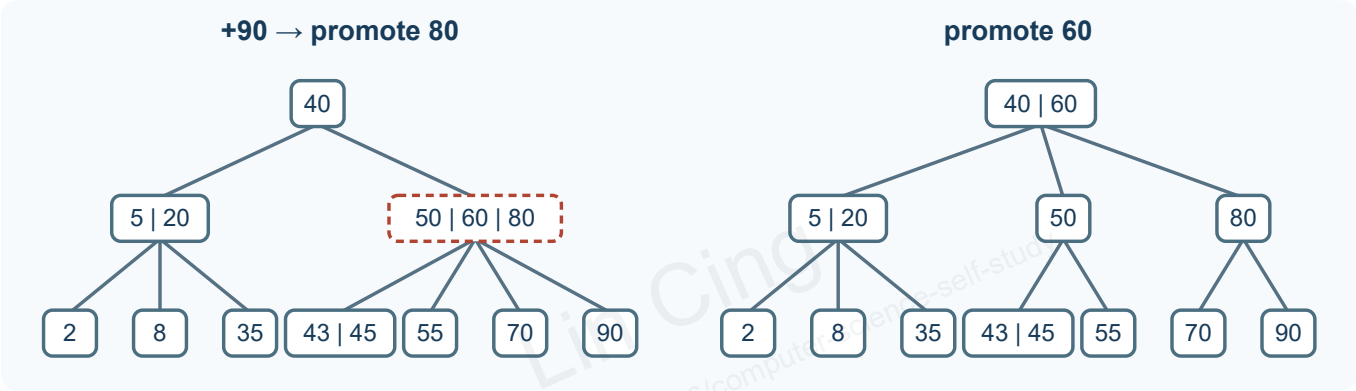
DS11-E08 • 兩種 B-tree 容量與分裂時機

order 3 最多 3 子、非根至少 1 鍵；t=2 最多 4 子、非根至少 1 鍵。order 3 滿節點只有 2 鍵，提升一鍵後只剩一鍵，不能使兩個新非根節點各有一鍵。插入後暫為 [a,b,c]，提升 b，左 [a]、右 [c]；若父也暫成三鍵，就重複向上分裂，根分裂時才增加高度。

DS11-E09 • 2–3 tree 的連續插入

插入 43 只改一葉，插入 2 提升 5 至 [20]，左內頁變 [5,20]。插入 90 先提升 80，再將父溢位的中間鍵 60 提至根。圖中虛線節點是處理過程的暫時溢位，不是完成後的合法節點。





最後中序為 2、5、8、20、35、40、43、45、50、55、60、70、80、90；共有 14 個互異鍵，各葉均在深度 2。非根節點皆有一或兩鍵。

DS11-E10 • 分裂時的磁碟頁讀寫

搜尋依序讀根 [60]、左內頁 [10]、葉 [5,8]，共 3 次頁讀取。插入後暫為 [2,5,8]，提升 5，原葉改成 [2]、新葉為 [8]，左內頁改成 [5,10] 並指向 [2]、[8]、[20]。因此寫入原葉、新葉及左內頁，共 3 次頁寫入，合計 6 次。根沒有改變，右子樹不讀不寫；新頁不另讀。若 buffer、頁配置或寫回政策不同，次數也可能不同。

DS11-E11 • 一般 order 的高度界

h≥1 時最少各層節點為 1、2、2q、…、2q^(h−1)，因此最少節點數為 1+2(q^h−1)/(q−1)。根至少 1 鍵，每個非根節點至少 q−1 鍵，最少鍵數是 1+(q−1)・2(q^h−1)/(q−1)=2q^h−1。全滿時各層為 1、m、…、m^h，最多節點為 (m^(h+1)−1)/(m−1)，每節點 m−1 鍵，最多鍵為 m^(h+1)−1。m=3、q=2、h=2 時依序為 7、13、7、26。h=0 時僅非空葉根，節點數固定 1，鍵數介於 1 與 m−1；公式同樣給出這四個端點。

參考資料與署名

Lin Cing，《資料結構》手寫筆記。 Pat Morin, Open Data Structures, § 14.2. PostgreSQL 18, B-tree implementation.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。
https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study